

Lock-Free Timer Scheduling With C++ Atomics

MAXIM GURSCHI

Lock-Free Timer Scheduling With C++ Atomics

Bloomberg
Engineering

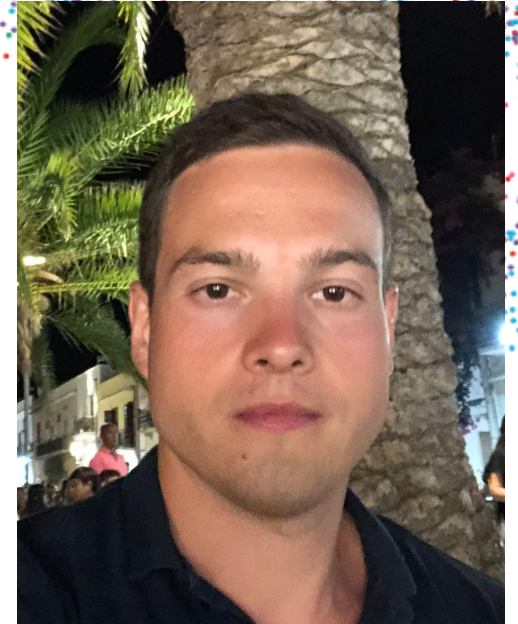
CppCon 2026
September 15, 2026

Maxim Gurschi
Lead Software Engineer

TechAtBloomberg.com

About me

- 15 years across front/middle/back office systems in financial institutions
- Specialised in building low-latency trading systems



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Taking it back

- [Mechanical Sympathy - Martin Thompson 2010](#)
- Understanding how software interacts with the hardware and operating system beneath it
 - CPU cache coherence
 - OS call overhead
 - Cost of mutex contention
 - Practical thread sleep/wakeup latency
 - Storage reuse
 - Avoiding IO on hot paths
 - All in the pursuit of fast, predictable execution

Data-driven applications

- A source of data (say market prices) updates periodically
- We want to display this data on our UI application
- Each update represents one **frame**
- Many client sessions may run concurrently
- Each session requires periodic updates
- We can decide to update the data several times per second

Data-driven applications

- Updating the display frequently provides little additional value to the user
 - This is further reduced if the UI contains many changing numbers
- At **4 updates/second**, each frame has a **250ms** budget. If computation takes **100ms**, the task is **idle** for the remaining **150ms**

Imagine thousands of independent instances of this UI

MadeUpCompany (MUC)

Generated using AI

REAL-TIME QUOTE

85.73 +2.84 (+3.43%) ▲

As of Jun 10, 2025 10:15:32 AM EDT

BUY

85.72

Size1,400

Bid Size (Total)5,800

Last Updated10:15:32 AM

SELL

85.74

Size1,000

Ask Size (Total)5,200

Last Updated10:15:32 AM

Open83.10

High85.98

Low82.72

Prev Close82.89

Volume1,512,600

Avg Volume1,205,400

Bid Size

Bid Orders

Bid Price ▼

Ask Price ▲

Ask Orders

Ask Size

1,400

4

85.72

85.74

4

1,000

1,100

3

85.71

85.75

2

1,200

1,000

2

85.70

85.76

3

900

800

2

85.69

85.77

2

800

700

1

85.68

85.78

1

700

600

2

85.67

85.79

2

600

500

1

85.66

85.80

1

500

400

1

85.65

85.81

2

400

300

1

85.64

85.82

1

300

200

1

85.63

85.83

2

200

Market Cap13.52B

P/E Ratio (TTM)25.61

EPS (TTM)3.62

Dividend (TTM)0.90

Dividend Yield1.05%

52W High86.40

52W Low58.10

Shares Outstanding156.10M

Beta (5Y)1.09

ExchangeNASDAQ

CurrencyUSD

Lot Size100

Tick Size0.01

Main concepts

- Schedule work periodically
- Support many independent work item instances
- Maintain regular execution



In one word...

TIMER

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Definitions

- Work item
 - An instance of a task that needs to be scheduled on a **Timer**
 - A user session that needs to be rendered periodically is a work item
 - Different work items can have different evaluation times
- Timer cycle (aka repeat period or frequency)
 - Time interval between successive evaluations of the same work item
- Timer Scheduler
 - The API used to schedule work items

Definitions

- **Producer**
 - The thread responsible for adding work items to the Timer Scheduler
- **Workers**
 - The pool of threads responsible for evaluating assigned work items
- **Work item batch**
 - All work items that a worker is evaluating during its execution cycle
- **Work item termination**
 - Removing the work item from the scheduler

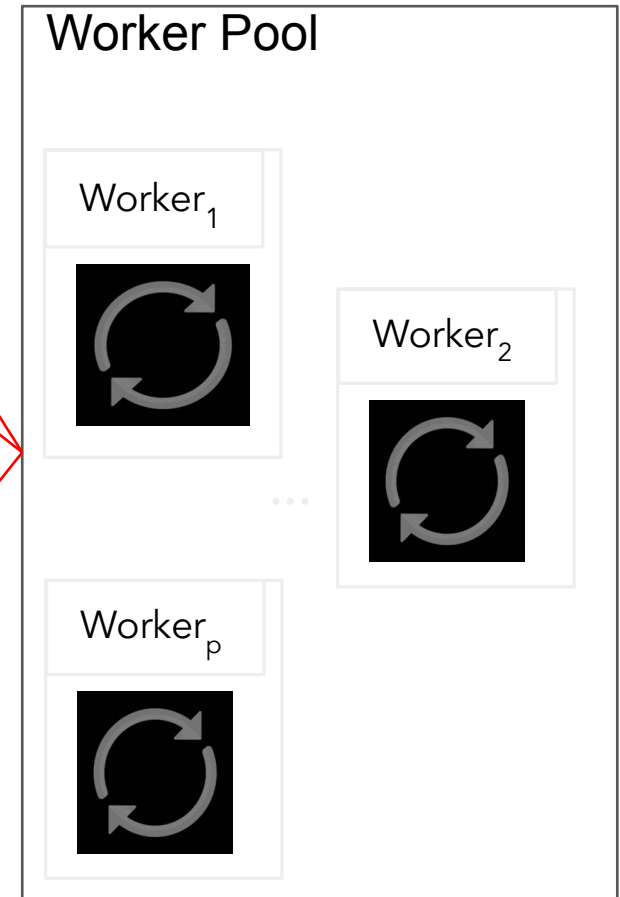
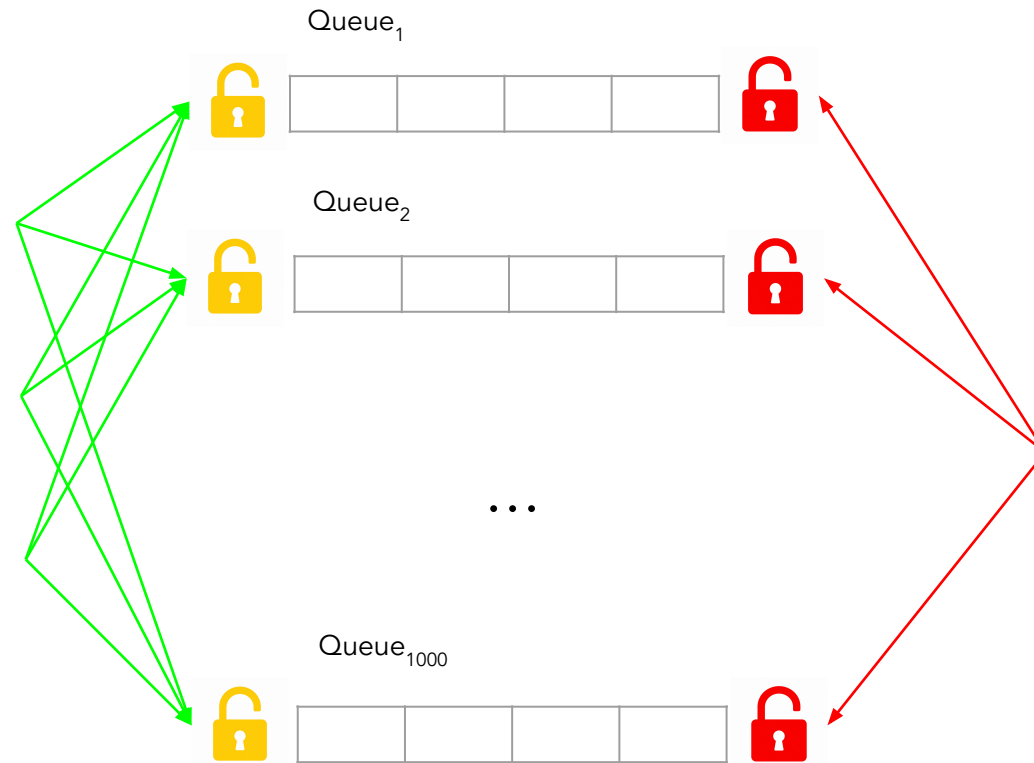
But what is the big deal?

- Give every task its own thread
 - Sleep or wait between executions
- Done? :)
- **Waste of resources**
 - The host has a finite number of CPU cores
 - Each thread carries stack and OS scheduling overhead
 - Most of the time is spent idling
 - More scheduling and context-switch overhead

Different Work Distribution Models

- **Work sharing**
 - Newly created work is proactively assigned to workers
- **Work stealing**
 - Idle workers pull work from other workers' queues
- [Blumofe & Leiserson](#) (1994)
- Today, I'll present a different model - **work giving**
 - **Workers aim to become idle by proactively giving away work to saturate other workers**

Variant of work stealing



1. Each session gets its own concurrent queue
2. The producer performs **$O(\text{number of sessions})$** recurring scheduling work
3. Multiple contention points
4. Work moves between workers, hurting locality

Some thoughts

1. These ownership-transfer operations can be implemented without locks using carefully scoped **atomics**
2. Instead of giving each session its own queue, use a shared preallocated node pool
3. When worker threads compete for the same queues, work items can move between cores from cycle to cycle, **hurting cache locality**
4. **There is one bottleneck** - the producer thread needs to keep rescheduling all the work items over and over
 - a. It also needs to continuously run, which **takes away another execution core**

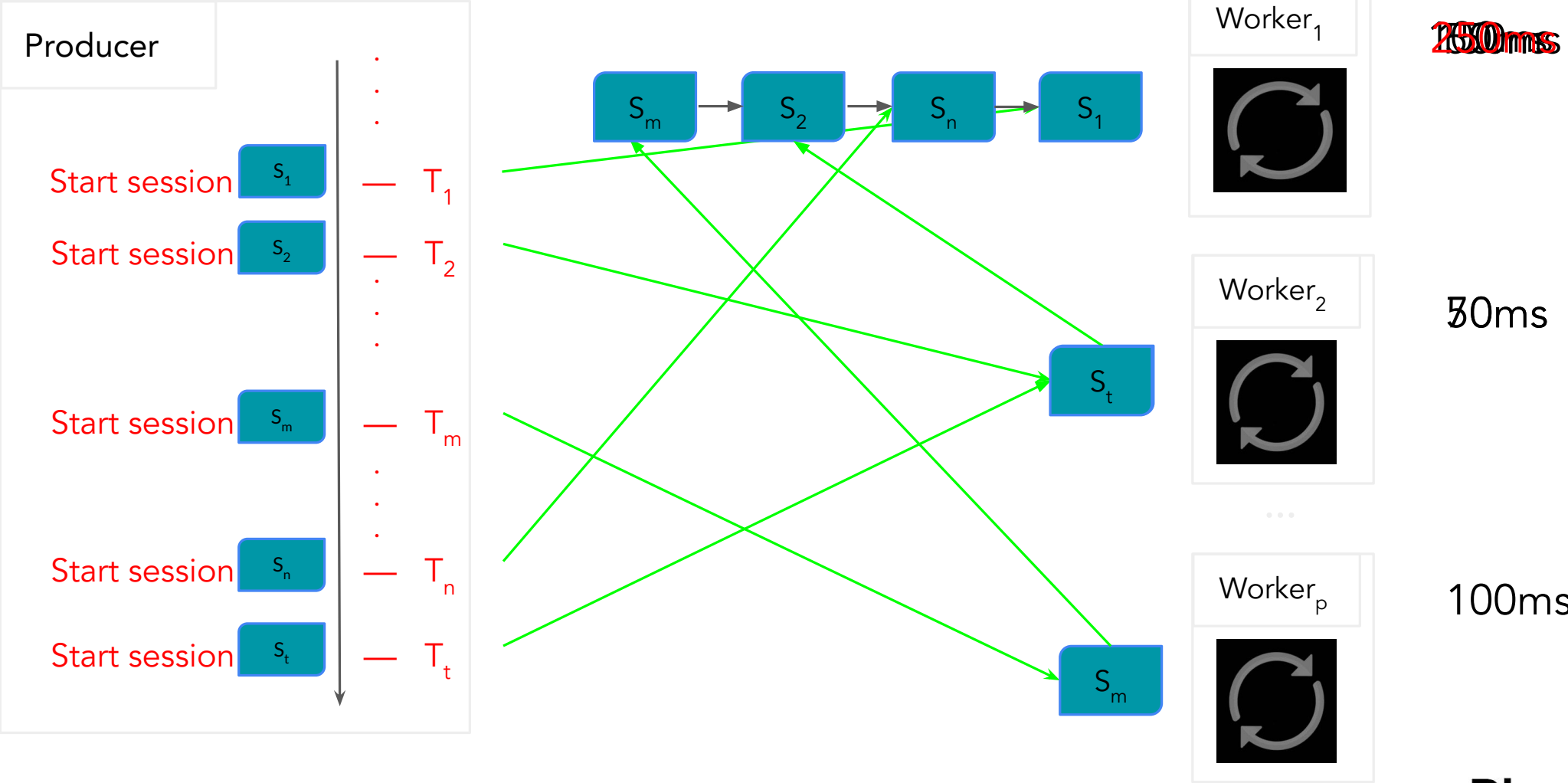
Requirements for the timer scheduler

- All work items in the same Scheduler instance have the same repeat cycle
- Represent work items **without type erasure** — use templates
- Avoid runtime allocations inside the scheduler
- Work is non-reentrant
 - No concurrent evaluation of the same work item
- Avoid a single bottlenecking producer
- **Schedule each work item once**
- **Workers reschedule subsequent executions themselves**

Requirements for the timer scheduler

- Adding, removing and migrating work items **should not require locks**
- Best effort to keep **average frequency stable**
 - Prepare the first frame as soon as possible
 - Aim to have a regular delay between subsequent frames
- Minimise constant work item core migration
- **Keep active workers busy - park surplus workers**

Work-Giving Timer Scheduler



Separation of Concerns

- Worker thread
 - `class BackgroundWorkerThread`
- Scheduler
 - `template<class Work> class TimerScheduler`
- **Lock-free data structure**
 - `class ConcurrentForwardLists`

BackgroundWorkerThread

- Encapsulates
 - `std::jthread`
 - `std::mutex`
 - `std::condition_variable`
- Essentially a while loop over a callback function
- Provides members to start/resume/pause the thread
 - Resume on demand
 - Resume when condition is true
 - Resume after a deadline
 - Resume on either condition or deadline

Disclaimer

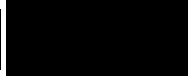
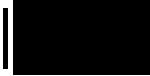
Thread parking **uses a condition variable**. However, as we will cover later, scheduling, migration, removal and reclamation are **lock-free** (*).

(*) Appropriate checks via `static_assert(std::atomic<T>::is_lock_free)`

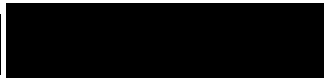
TimerScheduler

- Encapsulates
 - Scheduling parameters
 - How frequently to repeat the work
 - Time measurement tolerances
 - Runtime execution metrics
 - Maintains a short history of recent execution times
 - Uses this history to estimate expected execution time
- Schedules work items across worker threads
- Contains various heuristics to decide when to migrate work items across workers

Rationale for Work-Giving

- Worker A |  100ms|
- Worker B |  80ms|

Cycle **250ms** →

- Worker A |  180ms|
- Worker B | *idle* |

ConcurrentForwardLists

- Represents a fixed-size pool of forward nodes
 - Nodes provide storage for the scheduled work items
- Nodes are maintained in a free list owned by the **Producer**
- Once a work-item is emplaced, the **Producer** moves the corresponding node to the private list of a worker
 - Thus, the pool of nodes is, in a sense, distributed across sub-lists
- The **Producer** requests removal by **marking the node**
 - The owning worker **eventually observes** the mark and removes it

ConcurrentForwardLists: Sub-lists

- The node sub-lists contain non-overlapping nodes
- Each sub-list maps 1-to-1 to a **BackgroundWorkerThread** instance
- The sub-lists provide APIs to take a prefix of the list and prepend it to another sub-list
 - This feature is what enables work item migration between workers
- When the worker detects (**eventual consistency**) that a work-item is removed, it returns it to the **Producer's** reclamation list

ConcurrentForwardLists: Members

- Container of Nodes
 - Count **N** forward nodes (next pointer) - these provide storage for the work items
- **N** per-node removal flags
 - These are used to mark work items that are removed from the scheduler
- Producer local free list: `Node* head`
- Cross-thread reclamation list: `std::atomic<Node*> reclaimedHead`

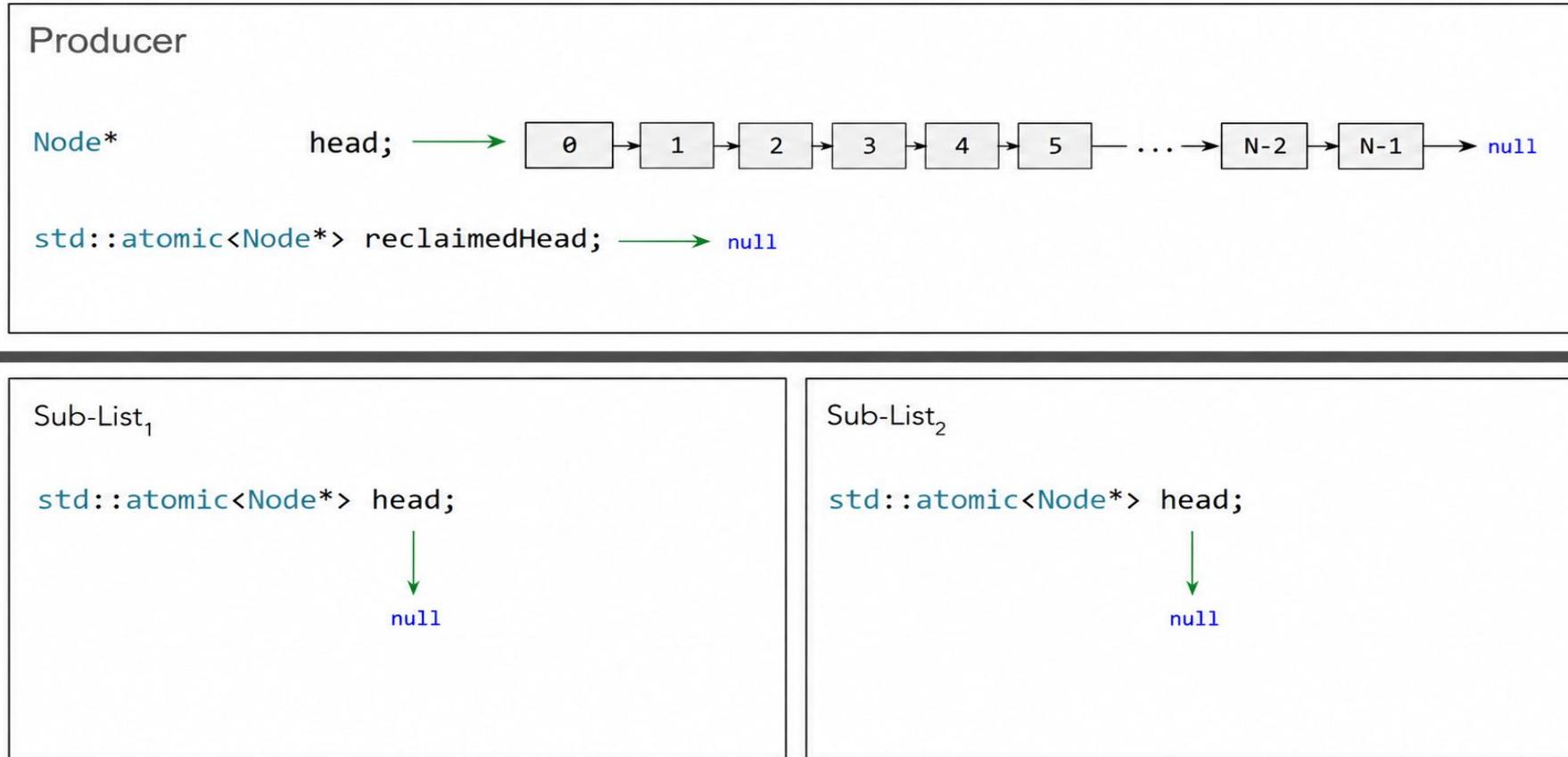
ConcurrentForwardLists: Members

- Container of size **W**
 - Stores the sub-lists
 - Each sub-list is essentially a `std::atomic<Node*>`
 - Padded to avoid **false sharing**
 - The expectation is that there will be one sub-list per **Worker**
- The design of the forward-list makes these operations cheap
 - Prepending/Extracting a prefix range
 - Modulo **CAS** retries, both are $O(1)$ in the common case (*)
 - (*) Range boundaries are known during sub-list traversal

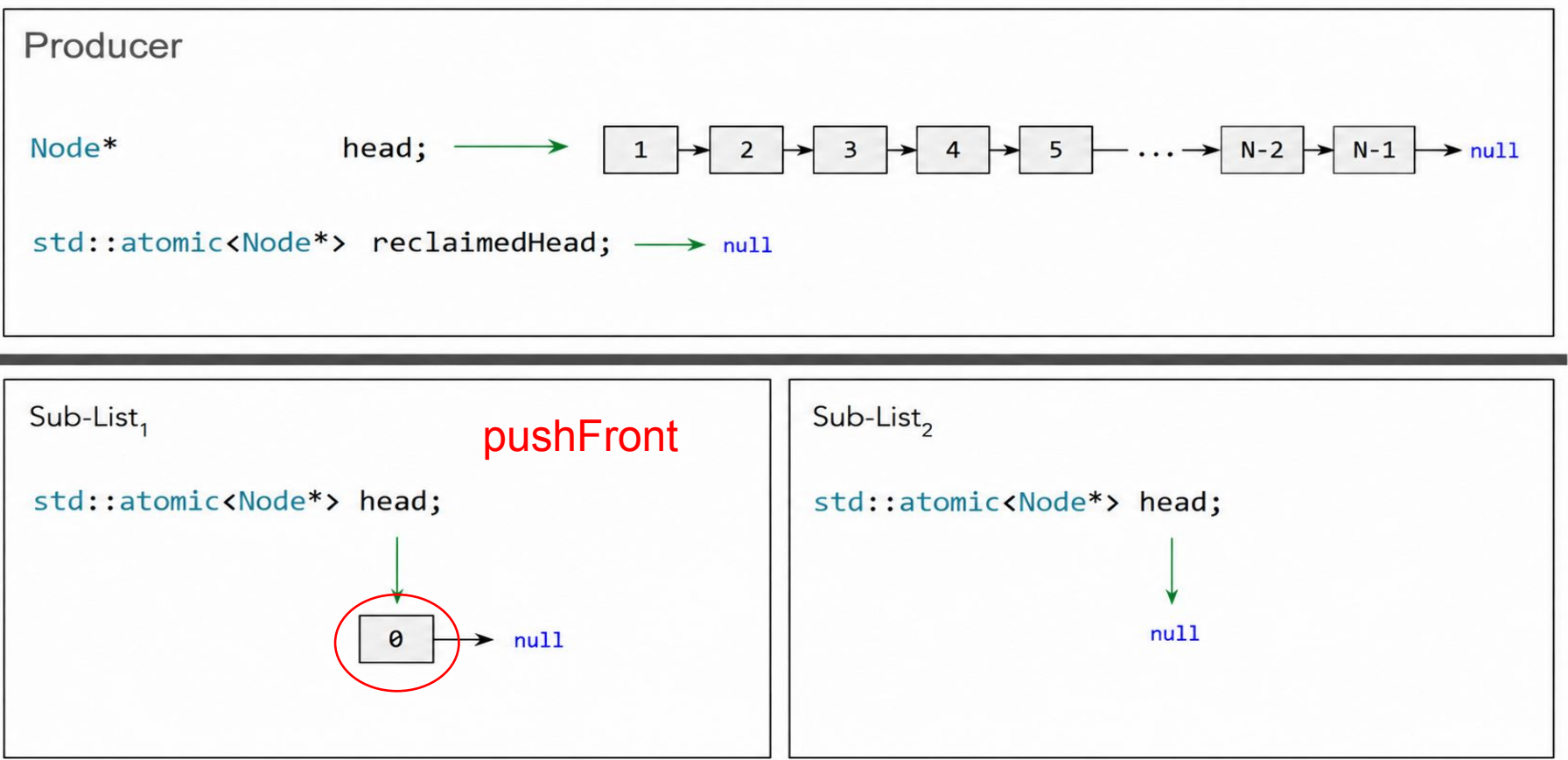
ConcurrentForwardLists: Invariants

- Every node has exactly one owner at all times
- Ownership may reside with a list or temporarily with the thread performing a transfer
- Nodes are relinked, never relocated

ConcurrentForwardLists: Initial state



ConcurrentForwardLists: 1st work-item



ConcurrentForwardLists: Sub-list *pushFront()*

```
5     void pushFront(Node* node) noexcept {  
6         node->next = head.load(std::memory_order_relaxed);  
7         while (!head.compare_exchange_weak(  
8             node->next,  
9             node,  
10            std::memory_order_release,  
11            std::memory_order_relaxed));  
12     }
```

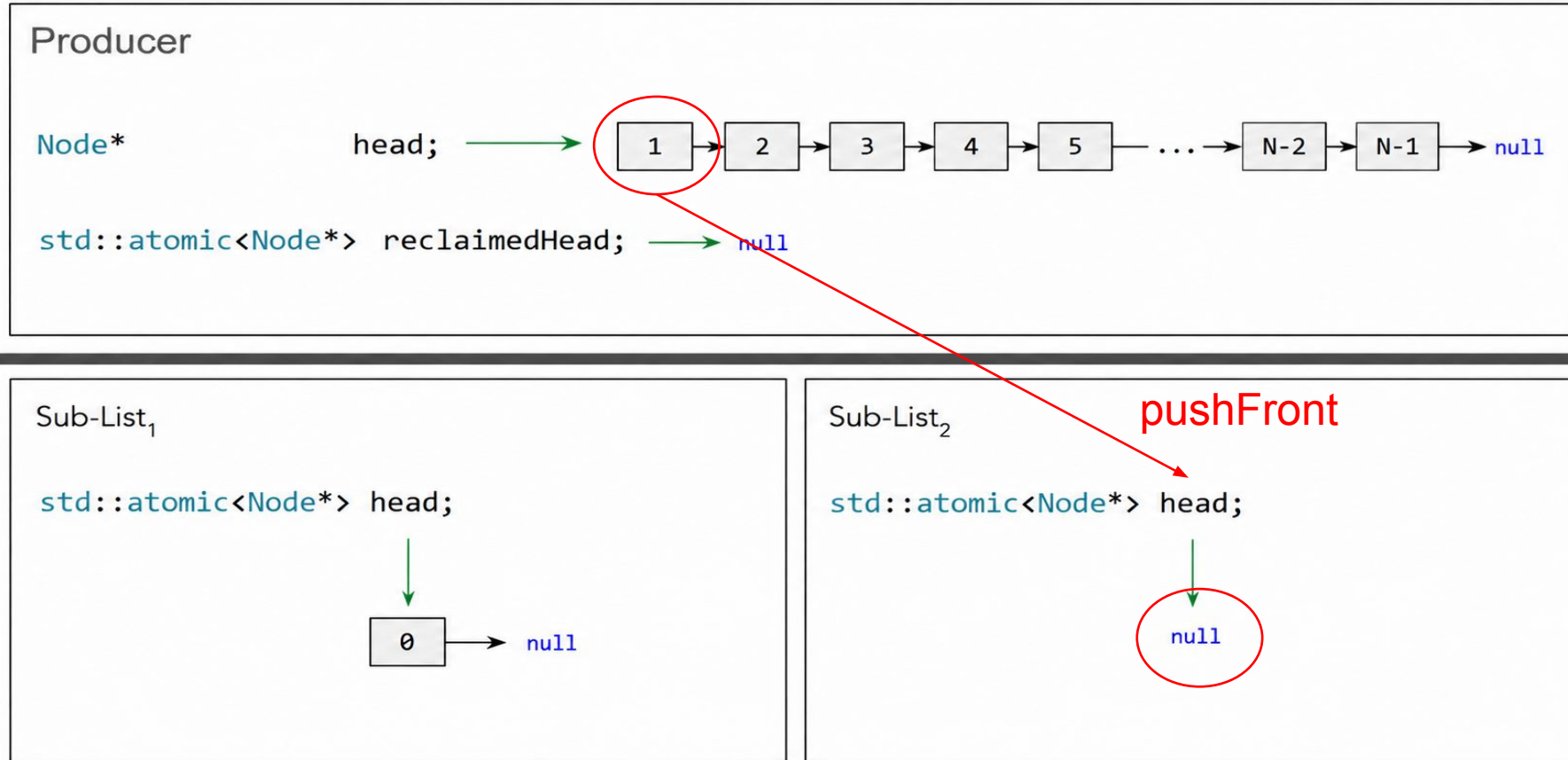
Both the Producer and Workers use this implementation

ConcurrentForwardLists: Sub-list *iterator begin()*

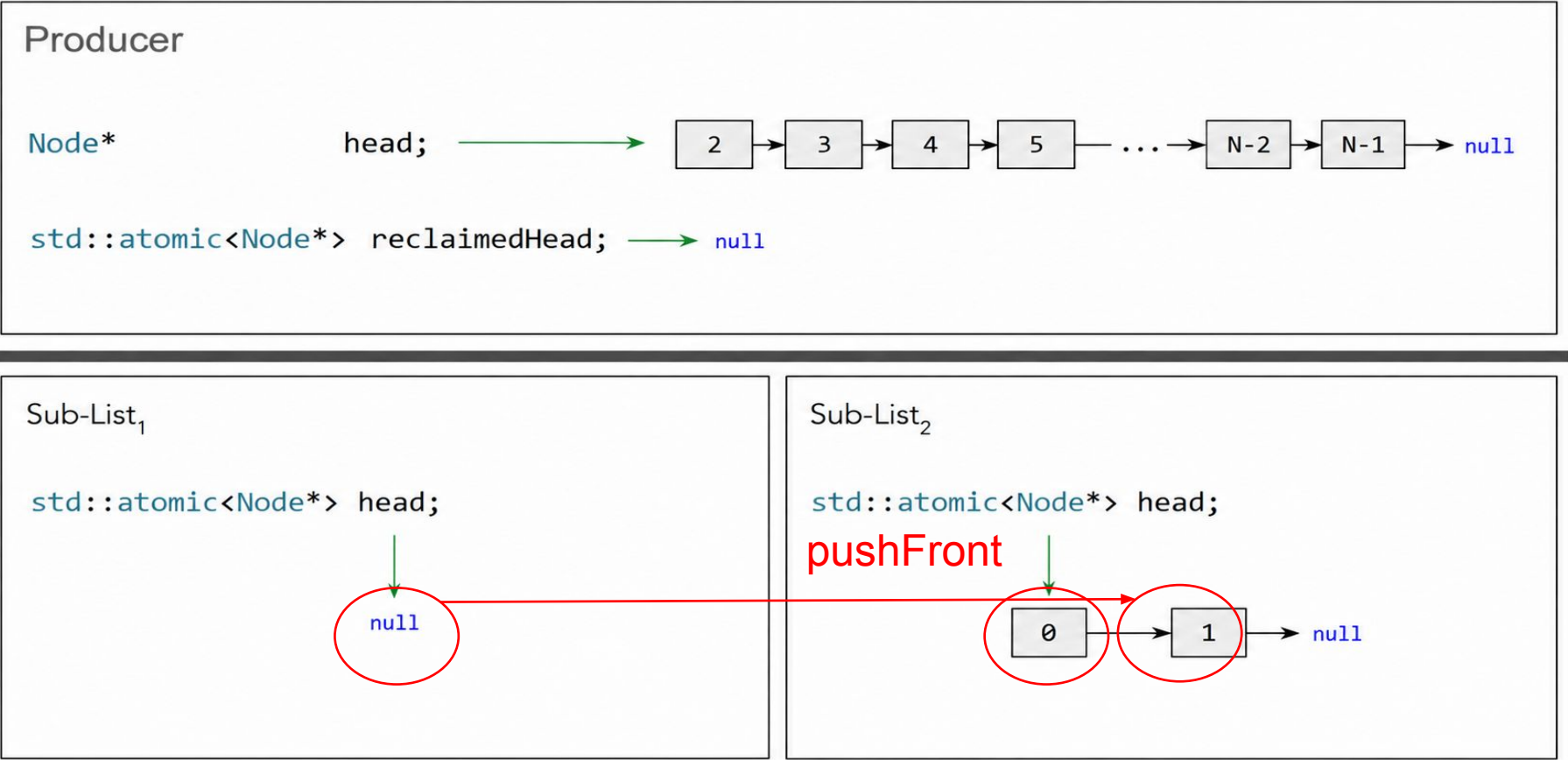
```
4 [[nodiscard]] iterator begin() noexcept {  
5     return iterator{head.load(std::memory_order_acquire)};  
6 }
```

1. Each `begin()` acquires a fresh snapshot of **head**
2. Concurrent prepends will **eventually** become visible on a subsequent cycle

ConcurrentForwardLists: 2nd work-item

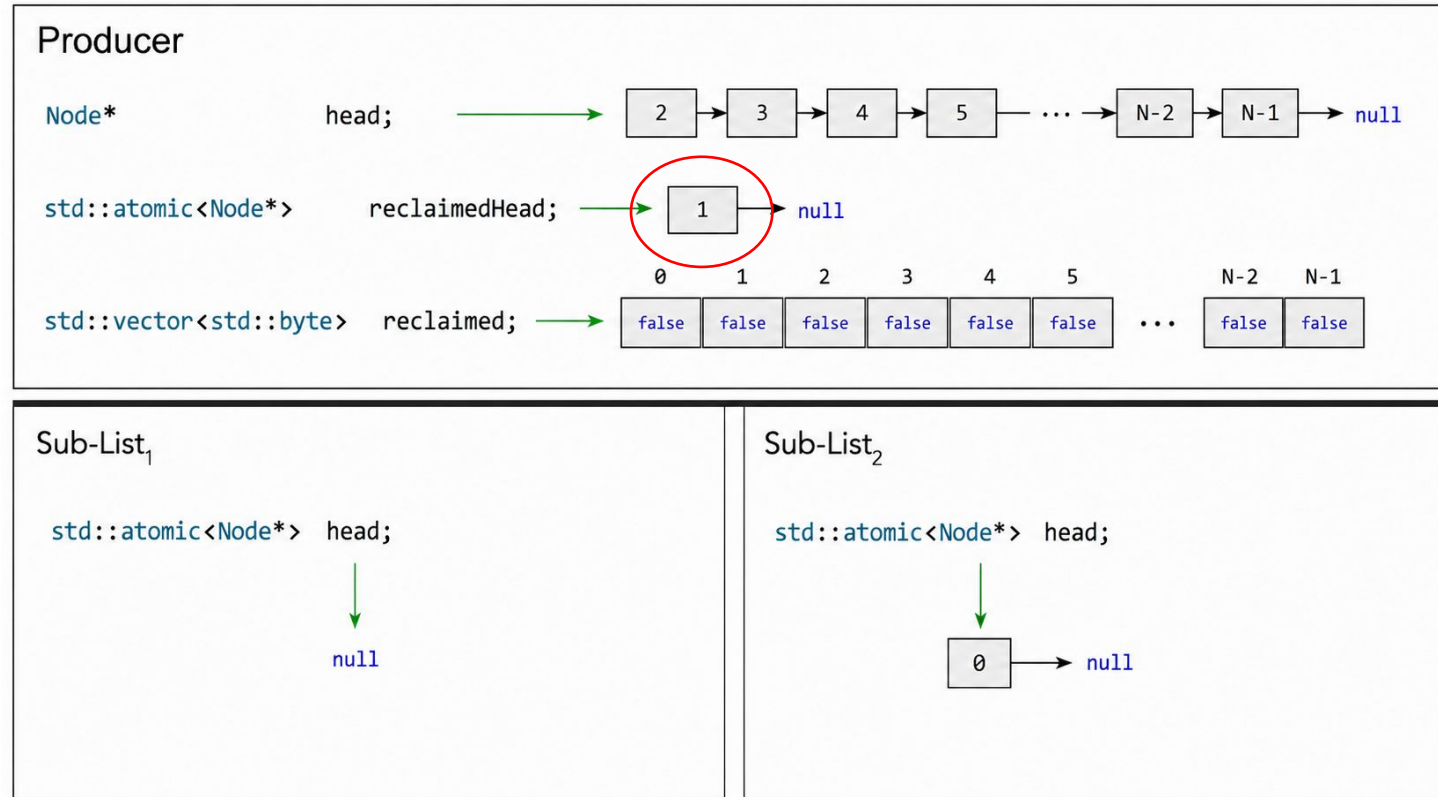


ConcurrentForwardLists: Merging work-items



ConcurrentForwardLists: Reclamation

1. Producer marks **node₁** for reclamation
2. Worker eventually observes the flag and removes it



(*) The **false/true** values are for illustration only - actual values are 0 and 1 respectively

ConcurrentForwardLists: *markReclamation()*

```
5     static constexpr std::byte FLAG_FALSE{0};
6     static constexpr std::byte FLAG_TRUE{1};
7
8     void markReclamation(std::byte& flag) noexcept {
9         std::atomic_ref{flag}.store(FLAG_TRUE, std::memory_order_release);
10    }
```

1. Each node has a corresponding **flag**
2. Producer marks the **flag**, and the worker eventually observes it
 - a. Worker-side check is just: `return std::atomic_ref{flag}.load(std::memory_order_acquire);`
3. Once the node is reclaimed, the **Producer** resets the **flag**

ConcurrentForwardLists: Worker reclamation

- Worker decides when to inspect the reclamation flags
- If the flag is set, the worker extracts the node

Depending on the relationship between the extracted node and the sub-list head, we have a number of possibilities:

1. Removed node is not the observed head → **unlink normally**
2. New nodes appeared ahead of it → **unlink normally**
3. Removed node is still head → **CAS required**

ConcurrentForwardLists: Worker reclamation

1. We illustrate the mentioned cases
2. Note, when traversing the forward list, we make the iterator keep track of the extracted node's previous node through **nodePrevious**
3. C++17 (P0418R2) allows a CAS to use release ordering on success and acquire ordering on failure

```
5 void extractNode(Node* node, Node* nodePrevious) {
6     Node* const next = node->next;
7     //1. Removed node isn't the observed head → unlink normally
8     if (nodePrevious != nullptr) {
9         nodePrevious->next = next;
10    } else {
11        Node* current = head.load(std::memory_order_acquire);
12        //2. New nodes appeared ahead of it → unlink normally
13        if (node != current
14            //3. Removed node is still head → CAS required
15            || !head.compare_exchange_strong(current,
16                                              next,
17                                              std::memory_order_release,
18                                              std::memory_order_acquire)) {
19            //Traverse current and relink.
20        }
21    }
22    //...
23 }
```

ConcurrentForwardLists: Producer node reuse

- When a producer needs to emplace values in the data-structure
 - Cheaply evaluates **head** pointer to see if there are free nodes
 - If none are present, **atomically** takes the entire reclaimed list
- If both pointers are null, then the **Producer** has no more free space to emplace values

ConcurrentForwardLists: Producer *full()*

```
6      [[nodiscard]] bool full() {  
7          return head == nullptr && (head = reclaimedHead.exchange(  
8                                  nullptr,  
9                                  std::memory_order_acquire)) == nullptr;  
10     }
```

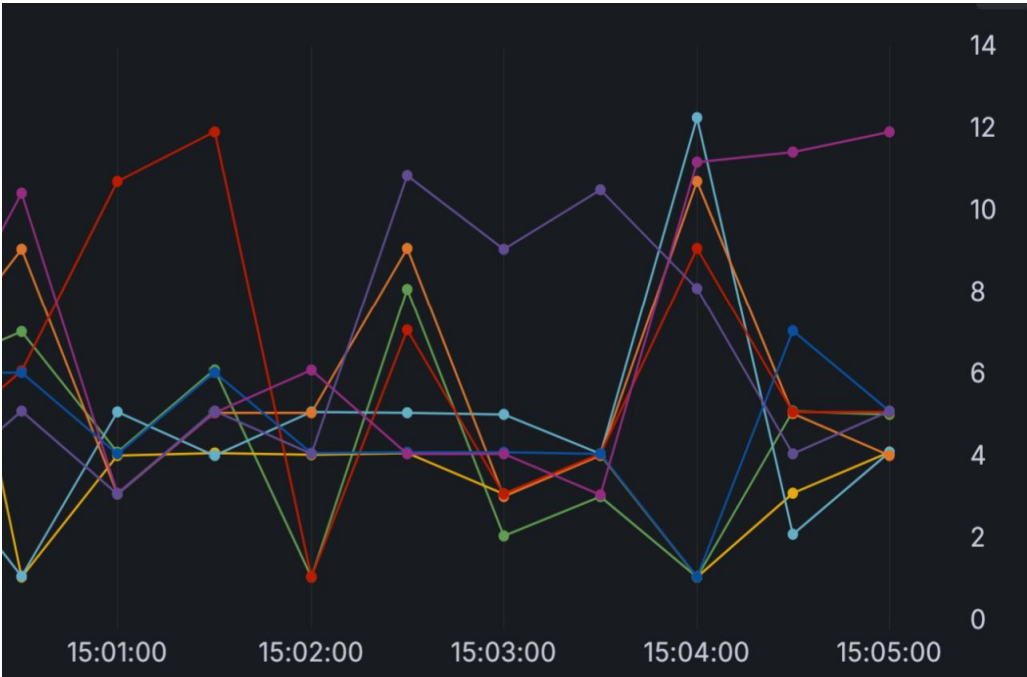
Performance Improvement

- **Work-Giving Timer Scheduler** vs. **Variant of work stealing**
- Scheduler overhead per complete cycle of 1,000,000 work items
- 8 worker threads in both implementations
- Repeat each work item every **250ms**
- Measure the time required to complete each full scheduling cycle
 - Scheduler time: **0.015s** vs **8.5s** (**>550x** reduction)
 - The baseline cannot keep up with the requested **250ms** cycle
 - Worker cores active during steady state - **1** vs. **8**
 - Process CPU usage - **5.8%** vs. **200%** (100% = one logical CPU)

Telemetry



Work-batch size distribution converges



Work-item migration remains limited

Takeaways

- The scheduler scales by moving ownership, not repeatedly rescheduling work
- Move work only when useful
- Don't synchronize information that doesn't need to be exact
- Hot-path ownership transfer and reclamation need not require locks - carefully scoped **atomic** operations are sufficient here
- Concentrate work onto fewer active cores

Bloomberg

Engineering

Thank you!

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



TechAtBloomberg.com

Bloomberg
Engineering